فازبندی فرانت‌اند

پروژه درس مهندسی نرم‌افزار | فاز اول، اسپرینت اول | برنامه مدیریت وظایف (Todo App)

نام برنامه: Done & Dusted | **شعار:** Get it done and dusted

زبان‌ها: فارسی (پیش‌فرض) و انگلیسی | **تم:** لایت و دارک

نام ریپو و پکیج: `done-and-dusted` | بدون علامت و فاصله

نقش: توسعه‌دهنده فرانت‌اند: رابط کاربری، تم، زبان‌ها، ولیدیشن سمت کلاینت و ارتباط با API

ابزارها: Next.js، React، TypeScript، Tailwind CSS، TanStack Query، next-intl

برنچ گیت: `frontend` | **پوشه:** `frontend/`

اصل کار موازی: در فاز ۰ قرارداد API با هم نوشته می‌شود. از فاز ۱ تا ۲ هر نفر مستقل کار می‌کند (با داده فیک (mock) کار می‌کنید و منتظر بک‌اند نمی‌مانید). در فاز ۳ دو بخش به هم وصل می‌شوند. فازها مرحله‌ای هستند و به روز مشخصی وابسته نیستند (کل پروژه حدود ۷ تا ۱۰ روز).

فاز ۰: آماده‌سازی مشترک (با هم‌تیمی)

هدف: قبل از جدا شدن، همه چیزهای مشترک روشن شود تا بعدش کسی منتظر دیگری نماند.

- ☐ Product Backlog بنویسید (عنوان، توضیح، اولویت، معیار پذیرش) و آیتم‌های «پشتیبانی از دو زبان فارسی/انگلیسی» و «تم لایت/دارک» را هم داخلش بگذارید
- ☐ Sprint Backlog بسازید: هدف اسپرینت، آیتم‌های انتخابی، تسک‌های فنی، تقسیم کار
- ☐ Definition of Done را تعیین کنید (کد زده شده، تست دستی شده، روی برنچ push شده، معیار پذیرش برآورده شده)
- ☐ قرارداد API، کدهای خطا و محدودیت‌ها را نهایی کنید (جدول زیر)
- ☐ تصمیم‌های مشترک: نام برنامه Done & Dusted، نام ریپو `done-and-dusted`، تاریخ نسخه فارسی شمسی باشد یا میلادی
- ☐ ساختار پوشه‌ها، `.env.example`، `.gitignore` و README اولیه روی `main` و ساخت برنچ‌های `frontend` و `backend`
- ☐ جلسه هماهنگی اول را برگزار و ثبت کنید

قرارداد API (مرجع مشترک)

```
POST /api/tasks   body: { "title": string, "description"?: string }
                  201 -> Task
                  400 -> { "error": "<ERROR_CODE>" }
GET  /api/tasks   200 -> Task[]   (newest first)

Task   = { id, title, description, status, createdAt }
status = "TODO"      (the only value in sprint 1)
Limits = title: 1-100 chars after trim, description: up to 500 chars
```

کد خطا	HTTP	چه زمانی؟
`TITLE_REQUIRED`	۴۰۰	عنوان ارسال نشده، خالی یا فقط فاصله است
`TITLE_TOO_LONG`	۴۰۰	عنوان بیش از ۱۰۰ کاراکتر است
`INVALID_INPUT`	۴۰۰	نوع داده نادرست یا توضیحات بیش از ۵۰۰ کاراکتر یا JSON خراب
`SERVER_ERROR`	۵۰۰	خطای غیرمنتظره سرور

API مستقل از زبان است: سرور فقط کد خطا می‌فرستد و فرانت آن را به فارسی یا انگلیسی ترجمه می‌کند. محدودیت‌های ۱۰۰ و ۵۰۰ پیشنهادی است و باید با هم‌تیمی نهایی شود، چون هر دو طرف باید یکسان رعایتشان کنند.

خروجی / معیار تکمیل: Backlogها آماده، قرارداد API و کدهای خطا مکتوب، ریپو و برنچ‌ها آماده.

فاز ۱: راه‌اندازی زیرساخت

هدف: پروژه فرانت با زبان‌ها، فونت‌ها و تم لایت/دارک بالا بیاید و زیرساخت نوشتن کامپوننت‌ها آماده باشد.

- ☐ ساخت پروژه Next.js با TypeScript و Tailwind داخل `frontend/`
- ☐ نصب `@tanstack/react-query` و ساخت `QueryClientProvider` در layout
- ☐ نصب و تنظیم `next-intl`: مسیرهای `[locale]` (پیش‌فرض `fa`)، middleware و فایل‌های `messages/fa.json` و `messages/en.json`
- ☐ بارگذاری فونت‌ها با `next/font/local`: وزیرمتن برای فارسی و Minako برای انگلیسی (مرجع ۲)
- ☐ layout با `lang` و `dir` درست (rtl برای فارسی، ltr برای انگلیسی)
- ☐ تعریف متغیرهای رنگ لایت و دارک با کلاس `dark` (مرجع ۱)
- ☐ دکمه تغییر تم و دکمه تغییر زبان در هدر؛ انتخاب تم ذخیره شود
- ☐ قرار دادن لوگوی لایت و دارک و فاویکون در `public/`
- ☐ تعریف تایپ `Task` مطابق قرارداد و لایه API (`getTasks`، `createTask`) با `NEXT_PUBLIC_API_URL` در `.env.local` و نمونه آن در `.env.example`
- ☐ داده فیک (mock) و سوییچ ساده بین mock و API واقعی

خروجی / معیار تکمیل: پروژه با `npm run dev` اجرا می‌شود، صفحه اصلی در هر دو زبان و هر دو تم درست نشان داده می‌شود.

فاز ۲: پیاده‌سازی قابلیت‌ها

هدف: Add Task، View Tasks و Input Validation سمت رابط کاربری، دو زبانه و در هر دو تم، کامل شود (با داده فیک).

- ☐ فرم افزودن وظیفه: عنوان (اجباری) و توضیحات (اختیاری) با `dir="auto"`
- ☐ ولیدیشن سمت کلاینت با همان محدودیت‌ها: عنوان خالی، فقط فاصله یا بیش از ۱۰۰ کاراکتر ثبت نشود؛ پیام از فایل ترجمه
- ☐ لیست وظایف: عنوان، توضیحات، بج وضعیت «انجام‌نشده» و زمان ایجاد با `Intl.DateTimeFormat`
- ☐ `useQuery` برای لیست، `useMutation` برای ثبت و `invalidateQueries` بعد از ثبت موفق
- ☐ تبدیل کد خطای سرور به پیام ترجمه‌شده (مثلاً `errors.TITLE_REQUIRED`)؛ کد ناشناخته به `SERVER_ERROR` نگاشت شود
- ☐ حالت‌های Loading، Error و Empty؛ پیام لیست خالی: Nothing left to dust. You're all done! و معادل فارسی آن
- ☐ کلاس‌های منطقی (Tailwind) `ms-`، `me-`، `text-start` و `rtl:` برای آیکون‌های جهت‌دار
- ☐ همه متن‌ها از فایل messages؛ اول فارسی کامل شود و ترجمه انگلیسی آخر این فاز اضافه شود
- ☐ بررسی هر کامپوننت در چهار حالت: فارسی و انگلیسی × لایت و دارک

خروجی / معیار تکمیل: با داده فیک می‌شود وظیفه اضافه کرد، لیست را دید و خطاها را مشاهده کرد، در هر دو زبان و هر دو تم.

فاز ۳: اتصال و یکپارچه‌سازی (با هم‌تیمی)

هدف: فرانت و بک واقعاً به هم وصل شوند و همه سناریوها از ابتدا تا انتها کار کنند.

- ☐ قطع داده فیک و وصل به API واقعی
- ☐ بررسی `NEXT_PUBLIC_API_URL` و رفع خطاهای CORS همراه نفر بک‌اند
- ☐ رفع ناهماهنگی‌ها: نام فیلدها، فرمت `createdAt`، کدهای خطا
- ☐ سناریو ۱: ثبت وظیفه موفق ← لیست خودکار به‌روز می‌شود
- ☐ سناریو ۲: عنوان خالی یا بیش از ۱۰۰ کاراکتر ← پیام خطای درست به زبان فعال
- ☐ سناریو ۳: refresh صفحه و ریستارت بک‌اند ← وظایف باقی می‌مانند (Persistence)
- ☐ سناریو ۴: وظیفه با عنوان فارسی و انگلیسی ثبت و درست نمایش داده شود
- ☐ سناریو ۵: تغییر زبان و تم وسط کار ← جهت، فونت، رنگ و داده‌ها درست بمانند
- ☐ مرج برنچ‌های `frontend` و `backend` به `main`

خروجی / معیار تکمیل: برنامه کامل روی `main` اجرا می‌شود و هر پنج سناریو درست کار می‌کند.

فاز ۴: مستندسازی و تحویل (با هم‌تیمی)

هدف: همه تحویلی‌های اسپرینت آماده و بررسی شوند.

- ☐ بخش فرانت README: نصب پکیج‌ها، تنظیم `.env.local` ، اجرا، محل قرار گرفتن فایل‌های فونت و توضیح زبان و تم
- ☐ مطمئن شوید فایل‌های فونت (و لوگوها) در ریپو هستند تا پروژه روی سیستم دیگران درست اجرا شود (لایسنس Minako را بررسی کنید)
- ☐ ریپو را از صفر کلون کنید و طبق README اجرا کنید
- ☐ Sprint Review: قابلیت‌های کامل‌شده، میزان تحقق هدف، بازخوردها، پیشنهاد ادامه
- ☐ Sprint Retrospective با ۴ پرسش و حداقل یک اقدام اصلاحی مشخص و عملی
- ☐ جمع‌بندی گزارش ۴ جلسه
- ☐ مرور چک‌لیست ۸ تحویلی

خروجی / معیار تکمیل: هر ۸ مورد تحویلی در ریپو موجود است و پروژه از صفر قابل اجراست.

جلسات هماهنگی (۴ جلسه)

جلسه	زمان	موضوع
۱	پایان فاز ۰	تعیین قرارداد API، کدهای خطا، Backlogها و تقسیم کار
۲	پایان فاز ۱	بررسی راه‌اندازی فرانت (i18n، فونت، تم) و مشکلات نصب
۳	پایان یا وسط فاز ۲	پیشرفت فرم و لیست، موانع قبل از اتصال
۴	پایان فاز ۳	نتیجه اتصال، باگ‌های باقی‌مانده، آماده‌سازی تحویل

هر جلسه ۱۵ تا ۲۰ دقیقه است و همان لحظه سه مورد ثبت می‌شود: پیشرفت نسبت به هدف، موانع، تصمیم‌ها و کارهای بعدی.

نکات مهم

- ☐ قرارداد API و کدهای خطا را بدون اطلاع هم‌تیمی عوض نکنید
- ☐ از API Routes خود Next.js استفاده نکنید؛ بک‌اند فقط Express است
- ☐ ولیدیشن کلاینت جایگزین ولیدیشن سرور نیست و فقط تجربه کاربر را بهتر می‌کند
- ☐ هیچ متنی را داخل کامپوننت هاردکد نکنید؛ همه از فایل ترجمه
- ☐ قابلیت‌های اضافه (ویرایش، حذف، تغییر وضعیت) فقط در Product Backlog با اولویت پایین نوشته شود
- ☐ مستندات و گزارش جلسات را همان روز بنویسید

دستورات گیت (برنچ شما)

```
git checkout frontend
git pull origin frontend
git add .
git commit -m "feat(frontend): add task form"
git push origin frontend
```

مرجع ۱: پالت رنگ لایت و دارک

نقش	لایت مود	دارک مود
پس‌زمینه صفحه	#FFF9FA	#160E10
سطح (کارت، فرم، اینپوت)	#FFFFFF	#211619
سطح ملایم (hover ردیف‌ها)	#FFF0F2	#2B1D21
بوردر	#F3D9DE	#3A2A2E
متن اصلی	#2A1A1E	#FFF9FA
متن ثانویه	#7A5C63	#C9A9B0
رنگ اصلی / دکمه اصلی	#FD1743	#FF4D6D
دکمه اصلی (hover)	#E0113A	#FF6B84
دکمه اصلی (کلیک)	#C20E32	#E63E5D
متن روی دکمه اصلی	#FFFFFF	#160E10
صورتی ملایم (بج، انتخاب)	#FFE3E8	#3A1620
غیرفعال (پس‌زمینه)	#F3E6E9	#2B1D21
غیرفعال (متن)	#B8A0A6	#6F5A5F
موفقیت (رنگ)	#1F9D55	#4ADE80
موفقیت (پس‌زمینه)	#E3F6EA	#12301F
هشدار (رنگ)	#B45309	#FBBF24
هشدار (پس‌زمینه)	#FFF1D6	#3A2C0B
خطا (رنگ)	#B91C1C	#F87171
خطا (پس‌زمینه)	#FDE8E6	#3B1618

- ☐ نام رنگ اصلی لایت: TRUE PINK و رنگ پس‌زمینه لایت: CHILL WHITE
- ☐ خطاها را همیشه با آیکون و متن نشان دهید، چون رنگ اصلی هم قرمز-صورتی است و ممکن است با خطا اشتباه شود
- ☐ متن روی دکمه در دارک مود تیره است (کنتراست بهتر از سفید)
- ☐ بج «انجام‌نشده» با رنگ هشدار (کهربایی) نمایش داده شود
- ☐ حلقه فوکوس: رنگ اصلی با ۴۰٪ شفافیت

```
:root  { --bg:#FFF9FA; --surface:#FFFFFF; --border:#F3D9DE; --text:#2A1A1E;
         --text-muted:#7A5C63; --primary:#FD1743; --primary-hover:#E0113A;
         --on-primary:#FFFFFF; --primary-soft:#FFE3E8; }
.dark  { --bg:#160E10; --surface:#211619; --border:#3A2A2E; --text:#FFF9FA;
         --text-muted:#C9A9B0; --primary:#FF4D6D; --primary-hover:#FF6B84;
         --on-primary:#160E10; --primary-soft:#3A1620; }
```

مرجع ۲: زبان، جهت و فونت

مورد	فارسی (پیش‌فرض)	انگلیسی
مسیر	`/fa`	`/en`
جهت صفحه	`dir="rtl"`	`dir="ltr"`
فونت	Vazirmatn	Minako (fallback: Vazirmatn)
تاریخ	`fa-IR` (شمسی، رقم فارسی)	`en-US`

```
// app/[locale]/layout.tsx
import localFont from "next/font/local";
const minako = localFont({ src: "../fonts/Minako-Regular.woff2", variable: "--font-minako" });
const vazirmatn = localFont({
  src: [{ path: "../fonts/Vazirmatn-Regular.woff2", weight: "400" },
        { path: "../fonts/Vazirmatn-Bold.woff2", weight: "700" }],
  variable: "--font-vazirmatn",
});
// <html lang={locale} dir={locale === "fa" ? "rtl" : "ltr"}
//       className={`${minako.variable} ${vazirmatn.variable}`}>

html[lang="fa"] body { font-family: var(--font-vazirmatn), sans-serif; }
html[lang="en"] body { font-family: var(--font-minako), var(--font-vazirmatn), sans-serif; }

new Intl.DateTimeFormat(locale === "fa" ? "fa-IR" : "en-US",
  { dateStyle: "medium", timeStyle: "short" }).format(new Date(task.createdAt));
```

- ☐ به‌جای `ml/mr`، `left/right` و `pl/pr` از `pe-`، `ps-`، `me-`، `ms-` و `text-start` استفاده کنید
- ☐ برای آیکون‌های جهت‌دار از `rtl:rotate-180` استفاده کنید
- ☐ روی عنوان، توضیحات و فیلدهای فرم `dir="auto"` بگذارید تا هر متن با جهت خودش چیده شود
- ☐ فایل‌های وزیرمتن (woff2) را از پکیج npm به نام `vazirmatn` بردارید؛ فایل Minako را خودتان دارید

لوگو

- ☐ دو فایل PNG شفاف: `logo-light.png` (رنگ `#FD1743`) و `logo-dark.png` (رنگ `#FF4D6D`)
- ☐ بر اساس تم فعال یکی از دو لوگو نمایش داده شود؛ لوگو به زبان صفحه ربطی ندارد
- ☐ یک نسخه ساده و مربعی برای فاویکون هم تهیه کنید